

Chapter 14: Functional Dependencies & Normalization

Fundamentals of Database Systems — Elmasri & Navathe (7th Ed.)

Comprehensive Exam Notes

1. Informal Design Guidelines for Relational Databases

Relational database design involves grouping attributes into 'good' relation schemas. There are four informal guidelines that serve as practical criteria.

1.1 Guideline 1 — Clear Attribute Semantics

Each tuple in a relation should represent ONE entity or relationship instance.

- Attributes of different entities (EMPLOYEE, DEPARTMENT, PROJECT) should NOT be mixed
- Only foreign keys should reference other entities
- Each relation should be easy to explain — the meaning of attributes should be clear

□ *A schema that mixes employee and department data into one relation makes it hard to interpret what a single tuple represents.*

1.2 Guideline 2 — Avoid Redundancy & Update Anomalies

Storing redundant information wastes storage and causes three types of update anomalies:

Anomaly Type	Description	Example (EMP_PROJ relation)
Insertion Anomaly	Cannot insert an entity without an unrelated entity existing	Cannot add a new project unless at least one employee is assigned to it
Deletion Anomaly	Deleting one entity unintentionally deletes another	Deleting the only employee on a project causes the project record to vanish
Modification Anomaly	Changing one fact requires updating many rows	Renaming project P1 from 'Billing' to 'Customer-Accounting' must be done for all 100 employees working on it

Example relation with anomalies:

```
EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
```

1.3 Guideline 3 — Minimize NULL Values

Relations should be designed so that tuples have as few NULL values as possible.

- Attributes that are frequently NULL should be moved to a separate relation (with the primary key)
- Reasons NULLs appear: attribute not applicable, value unknown, value known but unavailable

1.4 Guideline 4 — Avoid Spurious Tuples

Bad decompositions can produce meaningless (spurious) tuples when relations are joined back together.

- The lossless join (non-additive join) property must be guaranteed
- Property (a): Non-additivity — CANNOT be sacrificed
- Property (b): Preservation of functional dependencies — less critical, may be sacrificed if necessary

★ **EXAM TIP: Lossless join is non-negotiable. Dependency preservation is desirable but can sometimes be traded off.**

2. Functional Dependencies (FDs)

2.1 Definition

A set of attributes X functionally determines a set of attributes Y (written $X \rightarrow Y$) if:

For any two tuples t_1 and t_2 in $r(R)$: if $t_1[X] = t_2[X]$ then $t_1[Y] = t_2[Y]$

FDs represent real-world constraints on data. They are properties of the schema, not just of one particular state.

2.2 Examples of FDs

FD Notation	Meaning
$SSN \rightarrow ENAME$	Social security number uniquely determines employee name
$PNUMBER \rightarrow \{PNAME, PLOCATION\}$	Project number determines project name and location
$\{SSN, PNUMBER\} \rightarrow HOURS$	The combination of employee SSN and project number determines hours worked

2.3 Key Rule

If K is a key (primary or candidate) of relation R, then $K \rightarrow$ all other attributes in R.

2.4 Inferring FDs from Relation Instances

- Seeing repeated values for X with different Y values means $X \rightarrow Y$ does NOT hold
- If the same X always maps to the same Y, then $X \rightarrow Y$ MAY exist (not guaranteed — must be confirmed semantically)

Example — TEACH(Teacher, Course, Text):

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

- $Text \rightarrow Course$ MAY exist (each text maps to one course in this instance)

- Teacher → Course: RULED OUT (Smith maps to two different courses)
- Teacher → Text: RULED OUT
- Course → Text: RULED OUT (Data Structures maps to both Bartram and Horowitz)

□ Only violations can be definitively concluded from a single instance. Positive conclusions require semantic knowledge.

3. Normal Forms Based on Primary Keys

3.1 Normalization — Concept

Normalization is the process of decomposing 'bad' relations (with redundancy, anomalies) into smaller, well-structured relations.

A normal form is a condition based on keys and FDs that certifies a relation meets a quality standard.

3.2 Practical Use

- Designers typically normalize up to 3NF or BCNF in practice
- 4NF is rarely used; 5NF almost never
- Denormalization: storing a join of higher-NF relations as a lower-NF base relation — done deliberately for performance

3.3 Key Terminology

Term	Definition
Superkey	A set of attributes S such that no two tuples have the same value for S
Key (Candidate Key)	A minimal superkey — removing any attribute causes it to no longer be a superkey
Primary Key	One candidate key chosen as the principal identifier
Secondary Key	Any candidate key that is not the primary key
Prime Attribute	An attribute that is a member of ANY candidate key
Non-prime Attribute	An attribute that is NOT a member of any candidate key

3.4 First Normal Form (1NF)

A relation is in 1NF if all attribute values are atomic (indivisible).

1NF disallows:

- Composite attributes (e.g., Address broken into Street, City, Zip)
- Multivalued attributes (e.g., a set of locations stored in one cell)
- Nested relations (a relation inside a tuple)

Example — NOT in 1NF:

Dname	Dnumber	Dmgr_ssn	Dlocations (multi-valued)
Research	5	333445555	{Bellaire, Sugarland, Houston}

Dname	Dnumber	Dmgr_ssn	Dlocations (multi-valued)
Administration	4	987654321	{Stafford}

After converting to 1NF (one location per row — introduces redundancy):

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford

□ Most RDBMSs already enforce 1NF by default. Nested relations are handled by propagating the primary key into a new relation.

3.5 Second Normal Form (2NF)

Key Concepts:

- Full Functional Dependency: $X \rightarrow Y$ where removing ANY attribute from X destroys the dependency
- Partial Dependency: $X \rightarrow Y$ where some proper subset of X also determines Y

Example:

```
{SSN, PNUMBER} → HOURS      ← Full FD (neither SSN alone nor PNUMBER alone determines HOURS)
{SSN, PNUMBER} → ENAME       ← Partial FD (SSN alone → ENAME, so PNUMBER is unnecessary)
```

Definition:

A relation R is in 2NF if every non-prime attribute is FULLY functionally dependent on the primary key (no partial dependencies).

Example — EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation) violates 2NF:

- FD1: {Ssn, Pnumber} → Hours ← Full (OK)
- FD2: Ssn → Ename ← Partial dependency — VIOLATES 2NF
- FD3: Pnumber → {Pname, Plocation} ← Partial dependency — VIOLATES 2NF

After 2NF normalization, decompose into:

```
EP1(Ssn, Pnumber, Hours)      ← relationship data only
EP2(Ssn, Ename)               ← employee data
EP3(Pnumber, Pname, Plocation) ← project data
```

★ EXAM TIP: 2NF violations occur only when the primary key is COMPOSITE. If the PK is a single attribute, 1NF ⇒ 2NF automatically.

3.6 Third Normal Form (3NF)

Key Concept: Transitive Dependency

A transitive FD $X \rightarrow Z$ exists when $X \rightarrow Y$ and $Y \rightarrow Z$, where Y is not a key.

Example — EMP_DEPT(Ssn, Ename, Bdate, Address, Dnumber, Dname, Dmgr_ssn):

```
SSN → DNUMBER      (employee determines department number)
DNUMBER → DNAME     (department number determines department name)
SSN → DNAME         ← Transitive FD via DNUMBER — VIOLATES 3NF
```

Definition:

A relation R is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key.

After 3NF normalization of EMP_DEPT:

```
ED1(Ssn, Ename, Bdate, Address, Dnumber) ← employee
ED2(Dnumber, Dname, Dmgr_ssn)           ← department
```

NOTE on transitive dependencies:

The transitive dependency $X \rightarrow Y \rightarrow Z$ is only a problem when Y is NOT a candidate key.

Example: EMP(SSN, Emp#, Salary) — $SSN \rightarrow Emp\# \rightarrow Salary$ is NOT a violation because $Emp\#$ is a candidate key.

3.7 Informal Memory Aid for Normal Forms

Normal Form	Informal Rule
1NF	All attributes depend on THE KEY (atomic values, no repeating groups)
2NF	All attributes depend on THE WHOLE KEY (no partial dependencies)
3NF	All attributes depend on NOTHING BUT THE KEY (no transitive dependencies)

4. General Normal Form Definitions (Multiple Candidate Keys)

4.1 General 2NF

A relation R is in 2NF if every non-prime attribute is FULLY functionally dependent on EVERY candidate key of R (not just the primary key).

Example — LOTS(Property_id#, County_name, Lot#, Area, Price, Tax_rate):

- Candidate Key 1: {Property_id#}
- Candidate Key 2: {County_name, Lot#}
- FD: $County_name \rightarrow Tax_rate$ — violates 2NF because $County_name$ is part of a key, not the full key

Fix: decompose into LOTS1 and LOTS2:

```
LOTS1(Property_id#, County_name, Lot#, Area, Price)
LOTS2(County_name, Tax_rate)
```

4.2 General 3NF

A relation schema R is in 3NF if for every FD $X \rightarrow A$ in R, either:

- (a) X is a superkey of R, OR
- (b) A is a prime attribute of R

Condition (a) catches:

- Partial dependencies (2NF violations)
- Transitive dependencies (3NF violations)

Condition (b) allows FDs where a prime attribute is determined — these 'slip through' 3NF but are caught by BCNF.

Alternative 3NF definition — every non-prime attribute must be:

- Fully functionally dependent on every key of R
- Non-transitively dependent on every key of R

5. BCNF (Boyce-Codd Normal Form)

5.1 Definition

A relation R is in BCNF if for every FD $X \rightarrow A$ in R:

X must be a SUPERKEY of R

BCNF is stricter than 3NF. Every BCNF relation is in 3NF, but NOT every 3NF relation is in BCNF.

5.2 Normal Form Hierarchy

Normal Form	Basis	Strength
1NF	Atomic values	Weakest
2NF	1NF + no partial dependencies	↑
3NF	2NF + no transitive dependencies	↑
BCNF	Every determinant is a superkey	Stronger than 3NF
4NF	BCNF + no non-trivial MVDs	↑
5NF	4NF + no non-trivial JDs	Strongest

5.3 3NF vs. BCNF — The Classic Example

TEACH(Student, Course, Instructor) — assumptions:

- Each student takes a course from exactly one instructor
- Each instructor teaches only one course

FDs:

`fd1: {Student, Course} → Instructor`

fd2: Instructor $\rightarrow$ Course

- Candidate key: {Student, Course}
- fd2 violates BCNF because Instructor is NOT a superkey
- But TEACH is in 3NF because in fd2, Course (the dependent) is a prime attribute — condition (b) allows it

5.4 Achieving BCNF by Decomposition

Three possible decompositions of TEACH:

Decomposition	Relations	Lossless?
D1	{Student, Instructor} and {Student, Course}	No — spurious tuples generated
D2	{Course, Instructor} and {Course, Student}	No — spurious tuples generated
D3 ✓	{Instructor, Course} and {Instructor, Student}	YES — use this one

5.5 Non-Additive Join Test for Binary Decompositions (Property NJB)

A decomposition $D = \{R_1, R_2\}$ of R has the lossless join property if and only if either:

$(R_1 \cap R_2) \rightarrow (R_1 - R_2)$ is in F^+ , OR
 $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$ is in F^+

Applying to TEACH decomposition D3:

- $R_1 = \{\text{Instructor, Course}\}$, $R_2 = \{\text{Instructor, Student}\}$
- $R_1 \cap R_2 = \{\text{Instructor}\}$
- $R_1 - R_2 = \{\text{Course}\}$, $R_2 - R_1 = \{\text{Student}\}$
- Check: Instructor $\rightarrow$ Course ✓ (fd2 holds!) $\rightarrow$ D3 is lossless

5.6 General BCNF Decomposition Algorithm

Given relation R with FD $X \rightarrow A$ violating BCNF, decompose into:

(i) $R - A$ (all attributes except A)
(ii) $X \cup A$ (the violating FD's attributes)

Repeat for each resulting relation that is still not in BCNF.

For TEACH with violating FD Instructor $\rightarrow$ Course:

$R - \text{Course} = \{\text{Student, Instructor}\} \rightarrow (\text{Instructor, Student})$
 $X \cup \text{Course} = \{\text{Instructor, Course}\} \rightarrow (\text{Instructor, Course})$

This matches D3 — confirming the algorithm gives the lossless decomposition.

□ BCNF decomposition may lose functional dependency preservation — this is acceptable as long as the lossless join property is maintained.

6. Multivalued Dependencies (MVDs) and 4NF

6.1 Concept of MVD

An MVD $X \twoheadrightarrow Y$ on relation R means: if two tuples agree on X , then swapping their Y -values produces valid tuples too. Informally, X multi-determines Y independently of the other attributes.

Example — $EMP(ENAME, PNAME, DNAME)$ — an employee can have multiple projects and multiple dependents independently:

ENAME	PNAME	DNAME
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

- $ENAME \twoheadrightarrow PNAME$ (projects are independent of dependents)
- $ENAME \twoheadrightarrow DNAME$ (dependents are independent of projects)

A trivial MVD: $Y \subseteq X$, or $X \cup Y = R$ (the whole relation). Non-trivial MVDs cause redundancy.

6.2 Fourth Normal Form (4NF)

A relation R is in 4NF if for every non-trivial MVD $X \twoheadrightarrow Y$ in F^+ , X is a superkey of R .

Fix for EMP — decompose into two 4NF relations:

```
EMP_PROJECTS(ENAME, PNAME)
EMP_DEPENDENTS(ENAME, DNAME)
```

★ EXAM TIP: An MVD $X \twoheadrightarrow Y$ is a special case of a join dependency where $n = 2$. MVDs always come in pairs: if $X \twoheadrightarrow Y$ then $X \twoheadrightarrow Z$ (where $Z = R - X - Y$).

7. Join Dependencies (JDs) and 5NF

7.1 Join Dependency

A join dependency $JD(R_1, R_2, \dots, R_n)$ on R means that every valid state r of R can be losslessly decomposed into $R_1, R_2, \dots, R_n$:

$$\star (\pi_{R_1}(r), \pi_{R_2}(r), \dots, \pi_{R_n}(r)) = r$$

- An MVD is a special case of a JD where $n = 2$
- A trivial JD exists when one of the R_i equals R itself

7.2 Fifth Normal Form (5NF / PJNF)

A relation R is in 5NF (Project-Join Normal Form) if for every non-trivial $JD(R_1, R_2, \dots, R_n)$ implied by F , every R_i is a superkey of R .

Example — $SUPPLY(SNAME, PART_NAME, PROJ_NAME)$:

- This relation is in 4NF (no non-trivial MVDs)

- But it may have a JD: JD(R1, R2, R3) where R1={Sname, Part_name}, R2={Sname, Proj_name}, R3={Part_name, Proj_name}
- If so, SUPPLY is NOT in 5NF — decompose into R1, R2, R3

□ 5NF is rarely used in practice — discovering JDs in real databases with hundreds of tables is nearly impossible.

8. Quick Reference Summary — All Normal Forms

NF	Condition	What It Eliminates	Based On
1NF	All attribute values are atomic	Multi-valued & composite attributes, nested relations	Attribute atomicity
2NF	1NF + every non-prime attr fully depends on every key	Partial dependencies on composite keys	FDs + Keys
3NF	2NF + no non-prime attr transitively depends on any key	Transitive dependencies (via non-key intermediary)	FDs + Keys
BCNF	Every FD determinant is a superkey	Remaining anomalies involving prime attributes	FDs + Keys
4NF	BCNF + no non-trivial MVDs unless LHS is superkey	Multi-valued redundancy (independent multi-valued facts)	MVDs + Keys
5NF	4NF + no non-trivial JDs unless every Ri is a superkey	Join-dependency-based redundancy	JDs + Keys

9. Worked Examples for Exam Preparation

Example 1: Identify the Normal Form

Given: ORDER(Order_id, Customer_id, Customer_name, Product_id, Product_name, Qty)

FDs: Order_id → Customer_id, Customer_id → Customer_name, Product_id → Product_name, {Order_id, Product_id} → Qty

Primary Key: {Order_id, Product_id}

- Is it in 1NF? YES — all attributes are atomic
- Is it in 2NF? NO — Order_id → Customer_id is a partial dependency (Order_id is part of PK, Customer_id is non-prime)

Fix — decompose to 2NF:

```
ORDER_ITEMS(Order_id, Product_id, Qty)
ORDER_HEADER(Order_id, Customer_id)
CUSTOMER(Customer_id, Customer_name)
PRODUCT(Product_id, Product_name)
```

Example 2: Find Transitive Dependencies

Given: STUDENT(Std_id, Std_name, Dept_id, Dept_name, Advisor_id)

FDs: Std_id → Dept_id, Dept_id → Dept_name, Dept_id → Advisor_id

- Std_id → Dept_name — TRANSITIVE (via Dept_id) — violates 3NF
- Std_id → Advisor_id — TRANSITIVE (via Dept_id) — violates 3NF

Fix — decompose to 3NF:

```
STUDENT(Std_id, Std_name, Dept_id)
DEPARTMENT(Dept_id, Dept_name, Advisor_id)
```

Example 3: BCNF Check

Given: ENROLL(Student, Course, Instructor)

FDs: {Student, Course} → Instructor, Instructor → Course

Candidate keys: {Student, Course} and {Student, Instructor}

- In fd2 (Instructor → Course): Instructor is NOT a superkey → BCNF violated
- But Course is a prime attribute (part of candidate key {Student, Course}) → 3NF condition (b) holds → IN 3NF
- Conclusion: IN 3NF but NOT in BCNF

BCNF decomposition using violating FD Instructor → Course:

```
R1 = R - Course = {Student, Instructor}
R2 = Instructor U Course = {Instructor, Course}
```

★ EXAM TIP: When a relation is in 3NF but not BCNF, it means there is a FD where a prime attribute (not a superkey) determines another attribute. BCNF decomposition may sacrifice dependency preservation.

10. Key Formulas & Rules at a Glance

Concept	Formula / Rule
FD Definition	$X \rightarrow Y$ holds if $\forall t_1, t_2 \in r: t_1[X]=t_2[X] \Rightarrow t_1[Y]=t_2[Y]$
Key → All attrs	If K is a key of R, then $K \rightarrow R$ (K determines everything)
2NF check	No non-prime attr A should have: proper_subset_of_PK → A
3NF check	No non-prime attr A should have: $X \rightarrow Y \rightarrow A$ where Y is not a key
BCNF check	For every FD $X \rightarrow A$: X must be a superkey
NJB (Lossless test)	$D=\{R_1, R_2\}$ is lossless if $(R_1 \cap R_2) \rightarrow (R_1 - R_2)$ or $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$
BCNF decomposition	Violating FD $X \rightarrow A$: decompose into $(R - A)$ and $(X \cup A)$
4NF check	For every non-trivial MVD $X \twoheadrightarrow Y$: X must be a superkey
5NF check	For every non-trivial JD($R_1, \dots, R_n$): every R_i must be a superkey

— *End of Chapter 14 Notes* —